

Homework 2: Hidden XOR Structure and Simon's Algorithm

Quantum Computing Training 2026

This homework is about recognizing hidden XOR structure in a toy cryptographic construction and connecting it to Simon's algorithm. All additions of bit strings are XOR operations. We write this as $\oplus$. For example,

$$101 \oplus 011 = 110.$$

For bit strings $a, b \in \{0, 1\}^n$, the dot product $a \cdot b$ is computed modulo 2:

$$a \cdot b = a_1b_1 \oplus a_2b_2 \oplus \cdots \oplus a_nb_n.$$

Thus $a \cdot b = 0$ means the parity is even, and $a \cdot b = 1$ means the parity is odd.

Background: A toy encryption scheme

Let

$$P : \{0, 1\}^n \rightarrow \{0, 1\}^n$$

be a publicly known permutation. This means P rearranges the 2^n possible n -bit strings in a one-to-one and onto way. Since P is a permutation, it has an inverse P^{-1} . Let the secret key be an unknown bit string

$$k \in \{0, 1\}^n.$$

Define the encryption function with secret key k :

$$E_k(x) = P(x \oplus k).$$

The decryption function is

$$D_k(c) = P^{-1}(c) \oplus k.$$

The attacker knows P and has oracle access to E_k . The attacker's goal is to recover the secret key k .

A 3-bit example

For the warm-up questions, take $n = 3$ and

$$k = 101.$$

Let P be the following public permutation:

z	$P(z)$
000	011
001	110
010	000
011	101
100	111
101	001
110	100
111	010

For example, if $x = 011$, then

$$x \oplus k = 011 \oplus 101 = 110,$$

so

$$E_k(011) = P(110) = 100.$$

Problems

1. Warm-up: XOR, encryption, and decryption.

This problem checks that the toy encryption rule is clear before we look for hidden structure.

- (a) Using the 3-bit example above, compute $E_k(x)$ for each plaintext:

$$x = 000, \quad x = 001, \quad x = 111.$$

For each one, show both steps

$$x \mapsto x \oplus k \mapsto P(x \oplus k).$$

- (b) Prove that decryption recovers the original plaintext. That is, prove that for every $x \in \{0, 1\}^n$,

$$D_k(E_k(x)) = x.$$

Use the definitions

$$E_k(x) = P(x \oplus k), \quad D_k(c) = P^{-1}(c) \oplus k,$$

and the identity

$$a \oplus k \oplus k = a.$$

- (c) Explain why the public permutation P alone does not reveal the key k .

2. Building the Simon function.

The toy encryption scheme hides a shift by k . Simon's algorithm is designed to find hidden XOR structure, so we want to rewrite the attacker's information in a form that looks like Simon's problem.

Define

$$f(x) = E_k(x) \oplus P(x).$$

This function can be evaluated by an attacker because E_k is available through an oracle and P is public.

- (a) Substitute the definition of E_k into $f(x)$ and write $f(x)$ using only P , x , and k .
(b) Show that

$$f(x \oplus k) = f(x)$$

for every $x \in \{0, 1\}^n$. This shows that k is a period of f .

- (c) In Simon's problem, the promise is that there exists a secret string s such that

$$f(x) = f(y) \iff y = x \text{ or } y = x \oplus s.$$

In this cryptographic example, assume that P is chosen randomly, so that with high probability there are no accidental extra periods besides k . Under that assumption, explain why this f satisfies Simon's promise with $s = k$.

- (d) For the 3-bit example, compute the full table of

$$f(x) = E_k(x) \oplus P(x)$$

for all eight inputs $x \in \{0, 1\}^3$.

- (e) Check directly from your table that

$$f(x) = f(x \oplus 101)$$

for every input x . List the four paired inputs.

3. Running Simon's algorithm on the toy encryption scheme.

Now suppose the attacker turns the function $f(x) = E_k(x) \oplus P(x)$ into a standard quantum oracle

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle.$$

The first register has n qubits, and the second register has enough qubits to store the output of f .

- (a) Start with

$$|0^n\rangle |0^m\rangle.$$

After applying Hadamards to the first register, what is the state? Write your answer as a uniform superposition over all $x \in \{0, 1\}^n$.

- (b) Apply U_f to the state from part (a). Write the resulting state.
 (c) Suppose the second register is measured and the observed value is some output z . Simon's promise says that exactly two inputs are consistent with this output: x_0 and $x_0 \oplus k$. Write the post-measurement state of the first register.
 (d) Apply $H^{\otimes n}$ to the first register. Use the identity

$$H^{\otimes n} |x\rangle = \frac{1}{\sqrt{2^n}} \sum_{t \in \{0,1\}^n} (-1)^{t \cdot x} |t\rangle$$

to show that only strings t satisfying

$$t \cdot k = 0$$

can appear with nonzero probability.

- (e) Explain how repeated measurements give linear equations involving k . How many independent equations are needed to determine a nonzero $k \in \{0, 1\}^n$?
 (f) What hidden string does the quantum attack recover? Why does recovering this string break the encryption scheme?
 (g) Give the asymptotic number of quantum oracle queries needed by Simon's algorithm, in terms of n . Compare this with a brute-force classical search over all possible keys.

4. Qiskit implementation: recovering the key by running Simon's algorithm.

In this problem you will implement the attack on a small mystery instance of the toy encryption scheme. This instance uses $n = 3$. The secret key is an unknown nonzero string $k \in \{0, 1\}^3$. Your goal is to recover k by running Simon's algorithm.

The public permutation P is:

z	$P(z)$
000	010
001	110
010	011
011	000
100	001
101	111
110	100
111	101

You are also given oracle access to the encryption function E_k . For this small simulation, the oracle is specified by the following table:

x	$E_k(x)$
000	101
001	100
010	111
011	001
100	000
101	011
110	110
111	010

The hidden key k is not shown in the tables. You should recover it using the Simon procedure below.

- (a) In Python, store the two tables as dictionaries and construct the Simon function

$$f(x) = E_k(x) \oplus P(x).$$

Print the table of $f(x)$ for all $x \in \{0, 1\}^3$. This printout is a useful debugging check, but the goal of the problem is to recover k using the quantum sampling procedure.

- (b) Build a reversible Qiskit oracle for f :

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle.$$

The input register should have 3 qubits and the output register should have 3 qubits. One convenient way to build the oracle from a truth table is to loop over every input x , temporarily flip the input qubits whose bits are 0, and then use multi-controlled X gates to write the 1-bits of $f(x)$ into the output register.

The following helper function is one possible starting point. It assumes that bit strings are written in the usual order, such as "101", while Qiskit qubit lists are ordered from least significant to most significant. The `[::-1]` reversals keep those conventions consistent.

```
def xor_bits(a, b):
    return format(int(a, 2) ^ int(b, 2), f"0{len(a)}b")

inputs = [format(i, "03b") for i in range(8)]
```

```

P = {
    "000": "010", "001": "110", "010": "011", "011": "000",
    "100": "001", "101": "111", "110": "100", "111": "101",
}
E = {
    "000": "101", "001": "100", "010": "111", "011": "001",
    "100": "000", "101": "011", "110": "110", "111": "010",
}
f_table = {x: xor_bits(E[x], P[x]) for x in inputs}

def add_truth_table_oracle(qc, x_qubits, y_qubits, table):
    for x, fx in table.items():
        # Turn the control condition |x> into the all-ones condition.
        for q, bit in zip(x_qubits, x[::-1]):
            if bit == "0":
                qc.x(q)

        # XOR the bits of f(x) into the output register.
        for q, bit in zip(y_qubits, fx[::-1]):
            if bit == "1":
                qc.mcx(x_qubits, q)

        # Undo the temporary X gates on the input register.
        for q, bit in zip(x_qubits, x[::-1]):
            if bit == "0":
                qc.x(q)

```

- (c) Create the Simon circuit. Apply Hadamards to the input register, apply the oracle, apply Hadamards to the input register again, and then measure the input register. Run the circuit for many shots and plot a histogram of the measured input-register bit strings.

A possible skeleton is:

```

from qiskit import QuantumCircuit, transpile
from qiskit.providers.basic_provider import BasicSimulator
from qiskit.visualization import plot_histogram

n = 3
qc = QuantumCircuit(2*n, n)
x_qubits = [0, 1, 2]
y_qubits = [3, 4, 5]

qc.h(x_qubits)
add_truth_table_oracle(qc, x_qubits, y_qubits, f_table)
qc.h(x_qubits)
qc.measure(x_qubits, [0, 1, 2])

backend = BasicSimulator()
compiled = transpile(qc, backend)
counts = backend.run(compiled, shots=2048).result().get_counts()

```

```
print(counts)
plot_histogram(counts)
```

- (d) Each nonzero measured string t gives one linear equation

$$t \cdot k = 0 \pmod{2}.$$

Use the nonzero strings that appear in your histogram to solve for the hidden key k . Because Qiskit displays classical bits in a particular order, state clearly which bit-string convention you are using. If needed, reverse the displayed strings before writing the equations.

- (e) Once you have recovered k , verify it by checking that

$$E_k(x) = P(x \oplus k)$$

for all eight inputs in the mystery instance.

- (f) Use the recovered key to decrypt the ciphertext

$$c = 100.$$

That is, compute

$$D_k(c) = P^{-1}(c) \oplus k.$$

Bonus Problems

Bonus 1. A phase-oracle version of Simon sampling.

In the usual version of Simon's algorithm, the oracle writes $f(x)$ into an extra output register. Now consider a different kind of oracle: a phase oracle

$$O_f |x\rangle = (-1)^{f(x)} |x\rangle,$$

where

$$f : \{0, 1\}^n \rightarrow \{0, 1\}.$$

Suppose there is a nonzero string $c \in \{0, 1\}^n$ such that

$$f(x) = f(x \oplus c)$$

for every x .

This problem asks you to find one vector t satisfying $t \cdot c = 0$ without using any work qubits.

- (a) Start with $|0^n\rangle$ and apply $H^{\otimes n}$. Write the resulting state.
- (b) Apply the phase oracle O_f . Write the state as a sum over $x \in \{0, 1\}^n$.
- (c) Apply $H^{\otimes n}$ again. Show that the amplitude of an output string t is proportional to

$$\sum_{x \in \{0, 1\}^n} (-1)^{f(x)} (-1)^{t \cdot x}.$$

- (d) Pair each input x with $x \oplus c$. Use the promise $f(x) = f(x \oplus c)$ to show that if $t \cdot c = 1$, then the two contributions cancel.

- (e) Conclude that every string t measured with nonzero probability satisfies

$$t \cdot c = 0.$$

- (f) Compare this phase-oracle procedure with one round of the standard Simon algorithm. What information does each measurement give you, and why is one such vector t usually not enough to determine c ?

Bonus 2. More than one promised period.

Simon's original problem has one nonzero hidden period s . A natural generalization is that there may be many hidden periods.

Let $S \subseteq \{0, 1\}^n$ be a linear subspace over $\mathbb{F}_2$. This means that if $a, b \in S$, then $a \oplus b \in S$. Suppose a function $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$ satisfies the promise

$$f(x) = f(y) \iff x \oplus y \in S.$$

In words, f has the same value exactly on inputs that differ by an element of the period subspace S .

- (a) Explain why Simon's original promise is the special case where

$$S = \{0^n, s\}.$$

- (b) Suppose that after the oracle query and measurement of the output register, the first register is left in the state

$$\frac{1}{\sqrt{|S|}} \sum_{u \in S} |x_0 \oplus u\rangle.$$

Apply $H^{\otimes n}$ and show that measured strings t must satisfy

$$t \cdot u = 0 \quad \text{for every } u \in S.$$

The set of all such t is called the orthogonal subspace $S^\perp$.

- (c) If $\dim(S) = r$, what is $\dim(S^\perp)$? How many independent equations about S should you expect to learn from repeated samples?
- (d) Explain, at a high level, how one could recover the period subspace S rather than a single period string.
- (e) In a cryptographic setting, why might having many hidden periods be even worse than having one hidden period?